Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering (PCCoE)

Sector No. 26, Pradhikaran Nigdi, Pune – 411 044

An Autonomous Institute

Affiliated to Savitribai Phule Pune University (SPPU)

DEPARTMENT OF INFORMATION TECHNOLOGY

2026-27

A Report

On

DISTRIBUTED TASK SCHEDULER WITH DYNAMIC LOAD BALANCING AND FAULT TOLERANCE

By

Pushya Mitra: 123B1F080

Krushika Yelchalwar: 123B1F105

B.TECH (IT)

ABSTRACT

TaskGrid is a distributed task scheduling application designed to execute computational workloads across multiple independent worker processes. The application separates user interaction, coordination, task scheduling, execution, and monitoring. A central coordinator receives jobs, maintains task records, and assigns work to available worker nodes using workload-aware selection. Workers periodically send heartbeat messages, allowing the coordinator to maintain an operational view of the cluster. Task results and execution events are retained for monitoring and analysis.

The implementation uses Python and Flask for service endpoints, HTTP and JSON for inter-process communication, SQLite for task persistence, and HTML, CSS, and JavaScript for the web dashboard. Demonstrated workloads include CPU benchmarks and additional computation types such as prime-number analysis, Fibonacci calculations, statistics, and word frequency. The supplied execution evidence shows three connected workers, completed benchmark tasks distributed across the nodes, coordinator heartbeat traffic, and five passing computation unit tests. These observations establish the functioning of the local multi-process demonstration; controlled worker-failure recovery requires a separate failure-injection test to verify end to end.

Keywords: Distributed Computing, Task Scheduling, Load Balancing, Fault Tolerance, Heartbeats, REST API, Worker Nodes.

1. INTRODUCTION

Distributed computing allows independent processes or computers to cooperate through message exchange. Compared with a single execution process, a distributed worker model can improve throughput and operational flexibility when jobs can be processed independently. However, distributing work introduces challenges involving communication, workload allocation, coordination, monitoring, and recovery from process failures.

TaskGrid addresses these concerns through a coordinator–worker architecture. Users submit jobs through a browser interface; the coordinator records them and dispatches work to workers. The dashboard presents cluster activity, worker availability, task status, and execution results. This design is suitable for demonstrating the core principles of distributed systems in a laboratory environment.

1.1 Project objectives

• Implement independent coordinator and worker processes that communicate over HTTP.

• Allocate queued tasks among healthy worker nodes according to available capacity.

• Record task states, results, attempts, and operational events.

• Observe worker liveness through periodic heartbeat communication.

• Provide a browser-based interface for job submission and cluster monitoring.

• Evaluate computation correctness using automated tests and inspect live execution evidence.

2. PROBLEM STATEMENT AND SCOPE

A single processing server can become a bottleneck as the number of computational jobs increases. If the process becomes unavailable, tasks may remain incomplete and operators may lack sufficient visibility into the failure. The problem is to coordinate several independently running worker nodes, allocate work without unnecessarily overloading one node, and provide transparent information about execution status.

2.1 Proposed solution

The proposed application provides a central job coordinator and three worker services. Tasks enter a persistent queue, are assigned to available workers, and are monitored until completion or failure. The coordinator uses periodic heartbeats to track node availability. The dashboard polls application state and presents execution progress, worker slots, and event history.

2.2 Functional requirements

ID	Requirement	Expected behavior
FR-01	Task submission	Accept supported task types and JSON parameters.
FR-02	Scheduling	Dispatch queued jobs to available worker capacity.
FR-03	Worker monitoring	Show connected workers and active-task counts.
FR-04	Execution tracking	Show status, assigned worker, attempts and result.
FR-05	Activity audit	Display task and cluster events.
FR-06	Failure handling	Identify unavailable workers and support reassignment of unfinished work.

2.3 Non-functional requirements

The application should be understandable, reproducible on a Windows development machine, responsive to multiple task requests, and capable of preserving job history. Communication must use explicit service boundaries. The demonstration is intended for a trusted local development environment rather than deployment as a hardened internet-facing production service.

2.4 Scope and limitations

The submitted prototype demonstrates three local worker nodes on ports 5001–5003 and a coordinator on port 5000. The coordinator remains a single point of failure. The screenshots do not establish performance under production workloads, secure remote authentication, replicated storage, or automatic failover of the coordinator. Such capabilities are outside the verified scope of the present report.

3. SYSTEM ARCHITECTURE AND DESIGN

The system follows a coordinator–worker architecture. The browser is the client-facing presentation layer. The coordinator hosts the task API, persistent queue, scheduling logic, and cluster state. Workers expose independent execution endpoints and report liveness to the coordinator. A local SQLite database stores durable job and event information.

Figure 1. Logical architecture of TaskGrid

Layer	Component	Responsibility
Presentation	Web dashboard (browser)	Job submission, status, results and activity display
Coordination	Flask coordinator :5000	API, scheduling, persistence, worker health

Execution	Worker 1 :5001	Independent task processing
Execution	Worker 2 :5002	Independent task processing
Execution	Worker 3 :5003	Independent task processing
Storage	SQLite database	Task records, execution results and event history

3.1 Communication and coordination

The dashboard requests state from the coordinator using HTTP. Worker services send HTTP POST heartbeat messages to the coordinator, and the coordinator returns HTTP responses with status codes. The observed logs contain repeated POST /api/heartbeat and GET /api/state requests with HTTP 200 responses. These exchanges confirm active communication between independently running components.

3.2 Task lifecycle

A task is created with a unique identifier and a computation type. It enters the queue, is assigned to a worker, executes, and produces a result. The dashboard reports states such as queued, running, and completed. Execution attempts identify successive dispatches, which is important for detecting repeated execution after failure.

3.3 Load-balancing approach

The scheduler uses a least-loaded available-worker strategy: eligible workers are compared by reported active task counts relative to their configured capacity. When multiple workers have available slots, the coordinator distributes incoming tasks rather than relying on one fixed node. The supplied task table shows benchmark jobs completed by worker-1, worker-2, and worker-3, providing observable evidence of multi-worker distribution.

3.4 Fault-tolerance design

Heartbeat messages allow the coordinator to detect missing workers. The intended recovery path is to mark a worker unavailable, identify unfinished assignments, and return eligible jobs to the queue for another worker. Safe retries require attempt identification and side-effect-free or idempotent tasks. The screenshots supplied do not include a forced worker shutdown and subsequent reassignment; therefore this report describes the mechanism as a design feature rather than claiming a verified recovery test.

4. IMPLEMENTATION AND TECHNOLOGY

Technology	**Use in TaskGrid**
Python 3.13.3	Application logic and computation
Flask	Coordinator and worker HTTP services
HTTP / JSON	Inter-process communication and API messages
SQLite	Persistent task and event storage
HTML / CSS / JavaScript	Web dashboard and polling interface
Pytest 9.1.1	Automated computation tests
VS Code / Windows	Development and execution environment

4.1 Project modules

Coordinator service: Receives task submissions, tracks worker status, schedules jobs, and supplies dashboard state. Worker service: Runs as a separate process, receives computational jobs, reports status, and returns results. Core computation module: Implements supported algorithms independently of the web interface. Dashboard: Presents total tasks, completed tasks, running tasks, active workers, recent jobs, and audit events. Test module: Validates core computation behavior.

4.2 Observed development environment

The evidence was captured from a Windows PowerShell environment in the project folder D:\Final Year\TaskGrid_DCS_MiniProject. A Python virtual environment (.venv) was active. The test command executed was .\.venv\Scripts\python.exe -m pytest -v. The coordinator console displayed repeated successful heartbeat and state requests, consistent with the browser dashboard being active.

5. IMPLEMENTATION

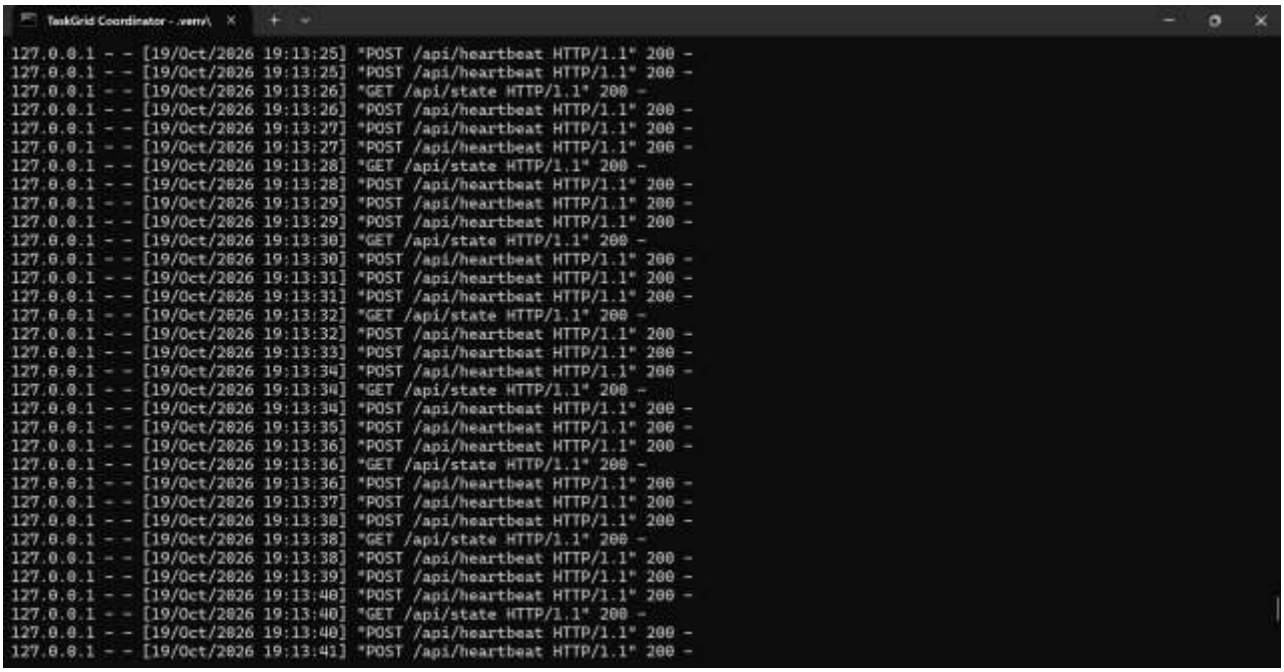


Figure 2. Coordinator console showing HTTP 200 heartbeat and state responses.

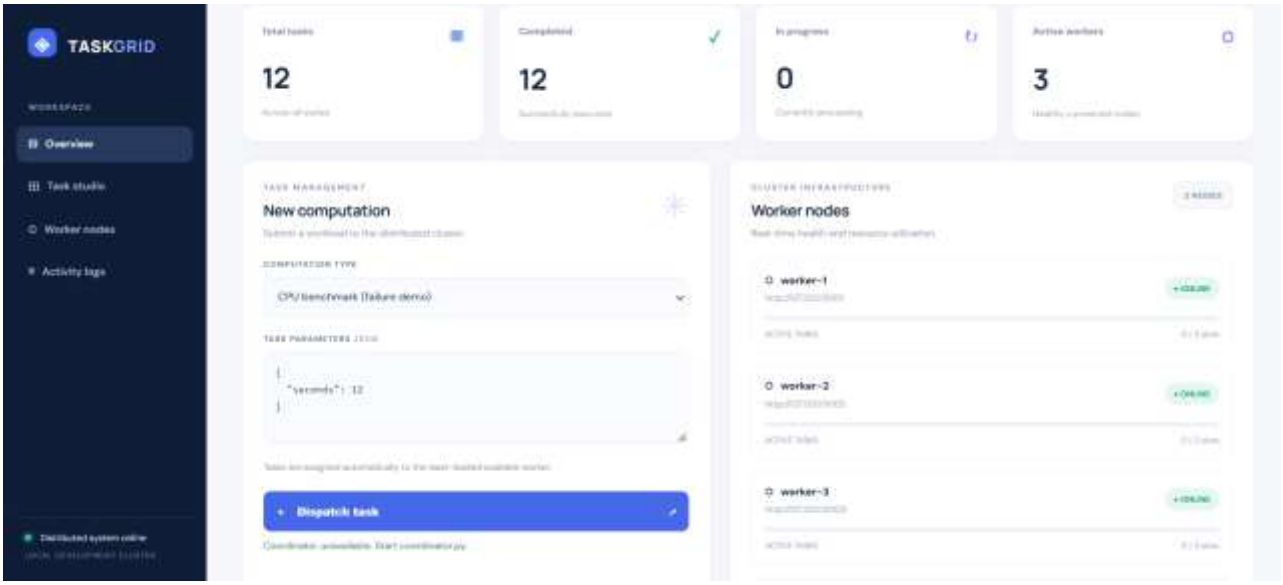


Figure 3. TaskGrid dashboard: summary metrics, task submission form and three online worker nodes.

The dashboard screenshot shows 12 total tasks, 12 completed tasks, zero tasks in progress, and three active workers. The three worker cards display worker-1, worker-2, and worker-3 as online, with endpoint ports 5001, 5002, and 5003. Each card indicates a configured capacity of two concurrent task slots.

The task form is configured for a CPU benchmark workload with a JSON parameter of 12 seconds. This screenshot documents the operator interface and visible cluster state at the time of capture. The small coordinator-unavailable message beneath the submission button is inconsistent with the live counters and online indicators; it may represent a transient UI message and should be checked during the final demonstration rather than ignored.

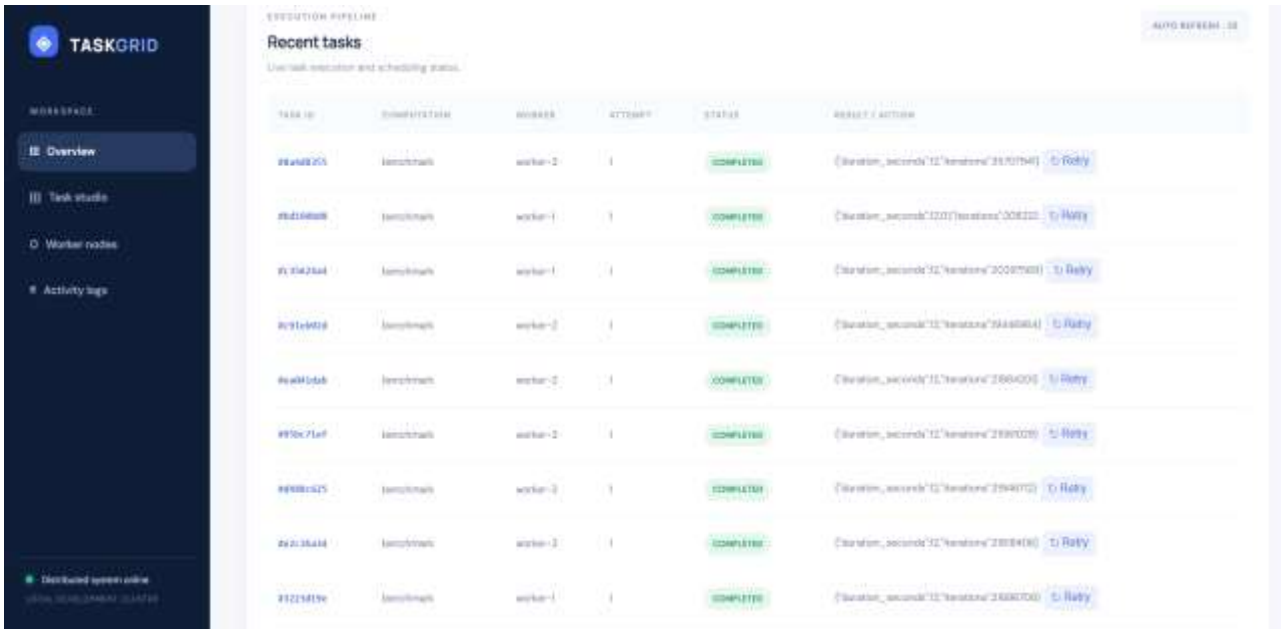


Figure 4. Recent tasks showing benchmark results, assigned worker nodes and completed status.

The recent-task table lists completed benchmark jobs with task identifiers, computation names, worker assignments, attempt numbers, statuses, and returned result objects. Visible assignments span all three worker nodes. Each displayed task is marked COMPLETED and shows attempt 1. This is evidence of successful multi-worker execution for the shown workload, not a measured throughput or fault-recovery benchmark.

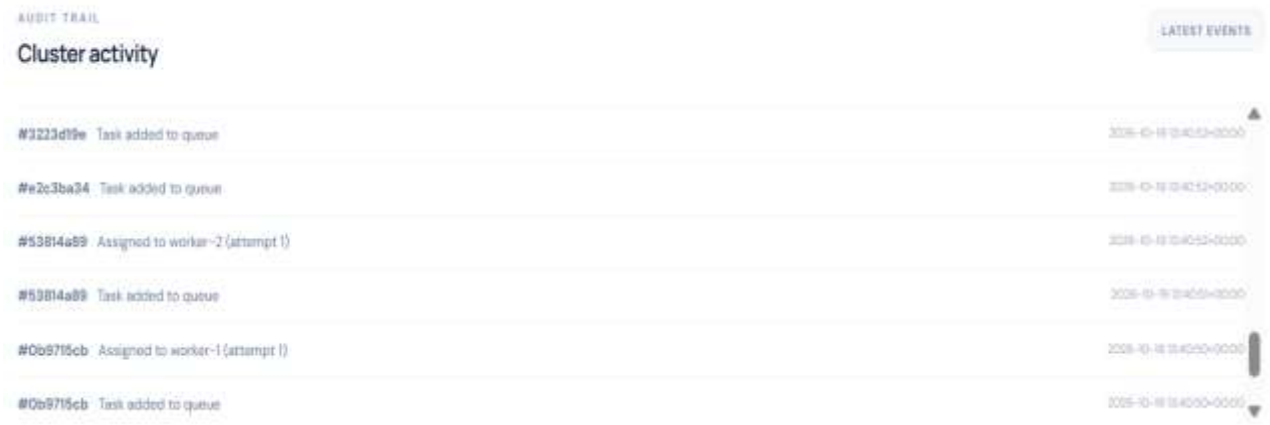


Figure 5. Cluster activity audit trail recording queue insertion and worker assignment events.

6. TESTING AND RESULTS

Automated testing was performed using Pytest within the project virtual environment. The screenshot shows five collected tests and five successful results, with a total reported duration of 0.29 seconds. The tests validate the computation module; they do not by themselves establish network recovery, persistence under crash, or end-to-end concurrency behavior.

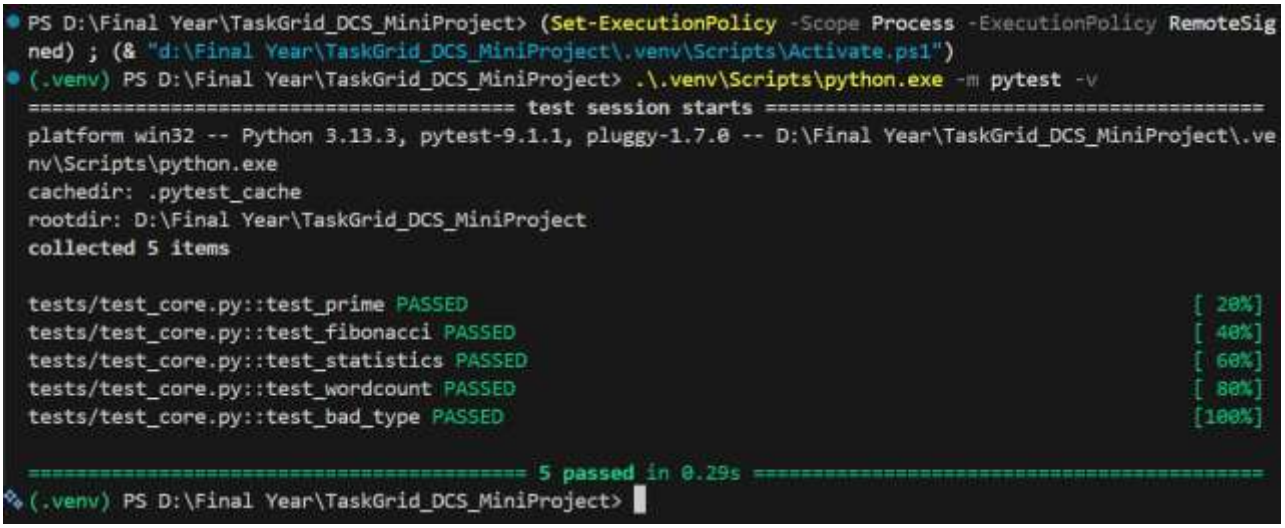


Figure 6. Pytest terminal output: five tests passed.

Test case	Observed outcome
Prime number computation	PASSED
Fibonacci computation	PASSED
Statistics computation	PASSED
Word count computation	PASSED
Invalid task type handling	PASSED

6.1 Result summary

At the time of dashboard capture, the cluster reported three healthy worker nodes and twelve completed tasks. The recent-task evidence shows benchmark executions distributed among the three workers, and the coordinator console shows repeated successful HTTP requests. Five computation tests passed. These findings support correct operation of the observed local demonstration.

6.2 Recommended final demonstration validation

To substantiate fault tolerance in the oral demonstration, submit a long-running benchmark, stop its assigned worker process, wait for the heartbeat timeout, and verify that the task is retried by another healthy node. Record the worker status change, task attempt count, and corresponding audit event. This controlled recovery test is recommended but is not represented in the supplied screenshots.

7. SDG MAPPING

SDG	Project contribution
SDG 9: Industry, Innovation and Infrastructure	TaskGrid demonstrates a software infrastructure pattern for distributing workloads, monitoring service health, and designing recovery mechanisms for computational services.

The project is mapped to United Nations Sustainable Development Goal 9 because it explores resilient digital infrastructure and practical innovation in computing systems. Its contribution is educational and technical: it demonstrates methods that can support reliable service processing. It does not claim to measure broader economic or environmental impact.

8. ADVANTAGES AND APPLICATIONS

The architecture separates scheduling from execution, enables independent worker processes, provides visibility into task progress, and supports expansion by adding worker nodes. Potential applications include background report generation, batch data processing, automated file analysis, scheduled computations, and classroom demonstrations of distributed coordination.

9. LIMITATIONS AND FUTURE SCOPE

The current local demonstration uses one coordinator and a small fixed worker cluster. It does not demonstrate coordinator high availability, distributed database replication, authenticated external access, or production-scale performance. Future improvements can include coordinator leader election, replicated persistence, secure service authentication, container deployment, queue prioritization, advanced scheduling metrics, and formal fault-injection testing.

10. CONCLUSION

TaskGrid demonstrates the practical organization of a distributed computing application through independent worker services, a coordinator, workload-aware task assignment, HTTP communication, and a monitoring dashboard. The supplied evidence records three online workers, twelve completed jobs, successful coordinator communication, and five passing computation tests. The design also incorporates worker heartbeat monitoring and a recovery pathway for unfinished jobs. Overall, the project provides a clear laboratory-scale foundation for understanding task scheduling, coordination, load balancing, and fault-tolerance principles.